

TheAILearner

Mastering Artificial Intelligence

Power Law (Gamma) Transformations

"Gamma Correction", most of you might have heard this strange sounding thing. In this blog, we will see what it means and why does it matter to you?

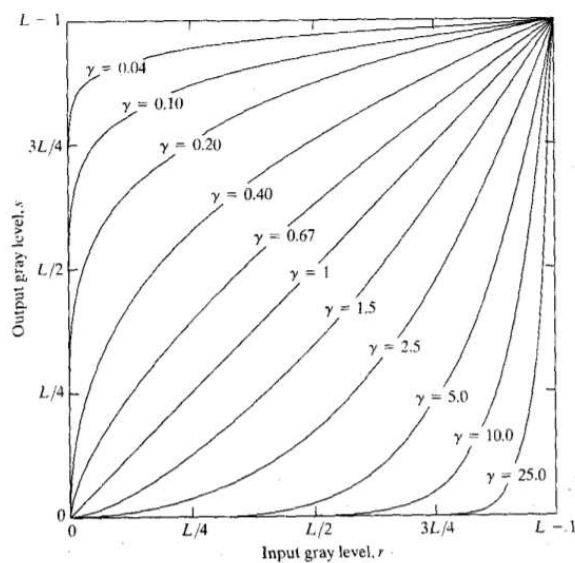
The general form of Power law (Gamma) transformation function is

$$s = c * r^\gamma$$

Where, 's' and 'r' are the output and input pixel values, respectively and 'c' and γ are the positive constants. Like [log transformation](#), power law curves with $\gamma < 1$ map a narrow range of dark input values into a wider range of output values, with the opposite being true for higher input values. Similarly, for $\gamma > 1$, we get the opposite result which is shown in the figure below

This is also known as gamma correction, gamma encoding or gamma compression. Don't get confused.

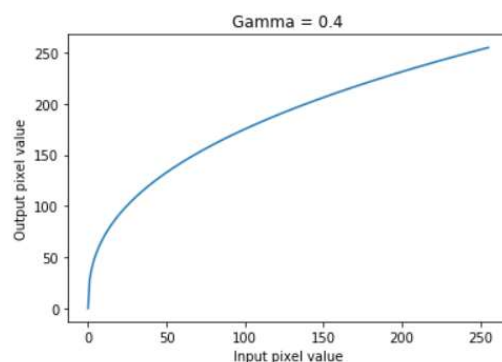
The below curves are generated for r values normalized from 0 to 1. Then multiplied by the scaling constant c corresponding to the bit size used.



All the curves are scaled. Don't get confused (See below)

But the main question is **why we need this transformation, what's the benefit of doing so?**

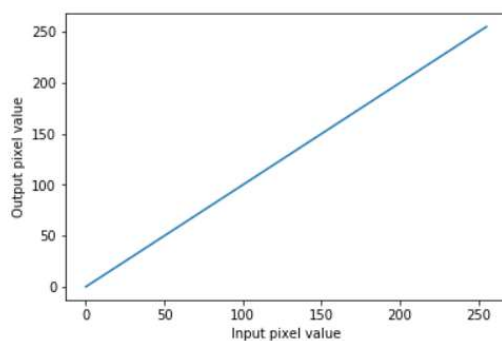
To understand this, we first need to know how our eyes perceive light. The human perception of brightness follows an approximate power function(as shown below) according to Stevens' power law for brightness perception.



See from the above figure, if we change input from 0 to 10, the output changes from 0 to 50 (approx.) but changing input from 240 to 255 does not really change the output value. This means that we are more sensitive to changes in dark as compared to bright.

You may have realized it yourself as well!

But our camera does not work like this. Unlike human perception, camera follows a linear relationship. This means that if light falling on the camera is increased by 2 times, the output will also increase 2 folds. The camera curve looks like this



So, where and what is the actual problem?

The actual problem arises when we display the image.

You might be amazed to know that all display devices like your computer screen have Intensity to voltage response curve which is a power function with exponents(Gamma) varying from 1.8 to 2.5.

Discover more

Machine Learning



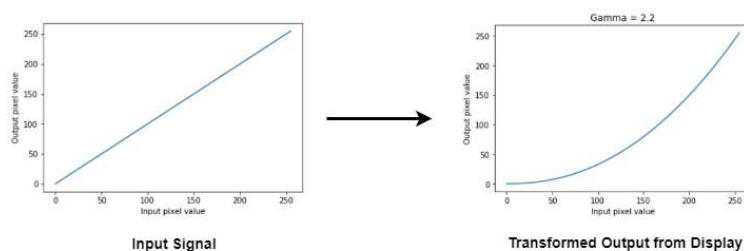
Machine Learning & Artificial Intelligence



Photo & Video Software

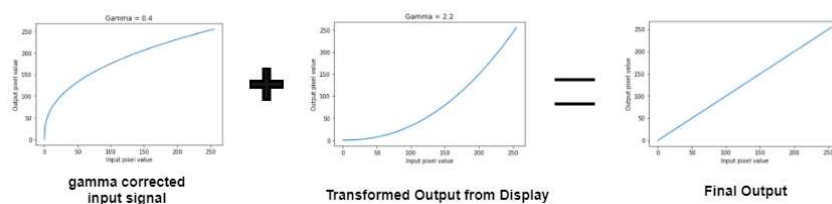


This means for any input signal(say from a camera), the output will be transformed by gamma (which is also known as **Display Gamma**) because of non-linear intensity to voltage relationship of the display screen. This results in images that are darker than intended.



To correct this, we apply gamma correction to the input signal (we know the intensity and voltage relationship we simply take the complement) which is known as **Image Gamma**. This gamma is automatically applied by the conversion algorithms like jpeg etc. thus the image looks normal to us.

This input cancels out the effects generated by the display and we see the image as it is. The whole procedure can be summed up as by the following figure



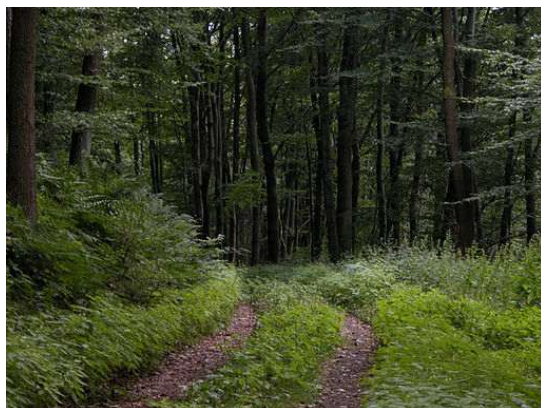
If images are not gamma-encoded, they allocate too many bits for the bright tones that humans cannot differentiate and too few bits for the dark tones. So, by gamma encoding, we remove this artifact.

Images which are not properly corrected can look either bleached out, or too dark.

Let's verify by code that $\gamma < 1$ produces images that are brighter while $\gamma > 1$ results in images that are darker than intended

```
1 import numpy as np
2 import cv2
3 # Load the image
4 img = cv2.imread('D:/downloads/forest.jpg')
5 # Apply Gamma=2.2 on the normalised image and then multiply by scaling constant (For 8 bit, c=255)
6 gamma_two_point_two = np.array(255*(img/255)**2.2,dtype='uint8')
7 # Similarly, Apply Gamma=0.4
8 gamma_point_four = np.array(255*(img/255)**0.4,dtype='uint8')
9 # Display the images in subplots
10 img3 = cv2.hconcat([gamma_two_point_two,gamma_point_four])
11 cv2.imshow('a2',img3)
12 cv2.waitKey(0)
13
```

The output looks like this



Original Image



Gamma Encoded Images

I hope you understand Gamma encoding. In the next blog, we will discuss Contrast stretching, a Piecewise-linear transformation function in detail. Hope you enjoy reading.

If you have any doubt/suggestion please feel free to ask and I will do my best to help or improve myself. Good-bye until next time.

Oxford Royale Yale University

Ad Oxford Royale ®

This entry was posted in Image Processing and tagged gamma corection, image processing, intensity transformation, opencv python on 26 Jan 2019 [<https://theailearner.com/2019/01/26/power-law-gamma-transformations/>] by kang & atul.

7 thoughts on “Power Law (Gamma) Transformations”



Nick Powers

30 Mar 2019 at 6:59 am

Great explanation. I would use a composition symbol instead of an addition symbol in the equation diagram though.



Camelot

10 May 2019 at 1:22 am

The best explanation I've found. Thank you!



Banshi

14 Sep 2019 at 10:55 pm

Intuitive. Thank you.



saurav

26 Sep 2019 at 5:06 pm

is it necessary to normalize the image before applying gamma transformation.

**kang & atul**

Post author

26 Sep 2019 at 6:23 pm

Hi sourav, it's not necessary but then you need to adjust the constant c. So, you can write $s=c*r^y$ where $c'=c/255^y$ and r range from 0-255(8-bit). Hope this helps.

**Olubunmi**

27 Aug 2020 at 8:50 pm

Thank you for this brilliant explanation. Please, how do I generate the power law curves using python.

**Waleed Ahmed**

5 Jun 2021 at 11:10 am

Can we take gamma value as negative?

[Privacy and cookie settings](#)

Managed by Google. Complies with IAB TCF. CMP ID: 300